

Ball Mapper: How the Algorithm Works

Ismail Soussi/Liam Grolle

Seminar Topological Data Analysis
Universität Münster

May 19, 2026

Outline

- 1 The Big Picture
- 2 Algorithm 1 – Greedy Cover
- 3 Algorithm 2 – Max-Min Cover
- 4 Algorithm 3 – Building the Graph
- 5 Algorithm 4 – Multi-Scale
- 6 Algorithm 4 – Multi-Scale Analysis
- 7 Dimension Estimation
- 8 All Algorithms Together

What does Ball Mapper produce? — Intuition

The core idea in one sentence

Take a large, complicated cloud of data points and turn it into a simple **graph** that shows the **shape** of the data.

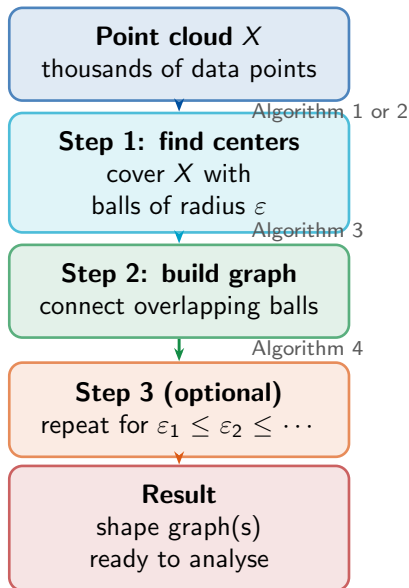
Think of it as a subway map:

- Data points = people standing in a town square.
- Balls of radius ε = umbrellas covering nearby people.
- A few people hold umbrellas so **everyone is covered**.
- Two umbrellas covering the **same person**?
⇒ Draw a **line** between them in the map.
- The result: a clean graph showing the crowd's shape.

Only one parameter needed

The radius ε – nothing else.

What does Ball Mapper produce? — Pipeline



Algorithm 1 – Greedy Cover: Concept

The task

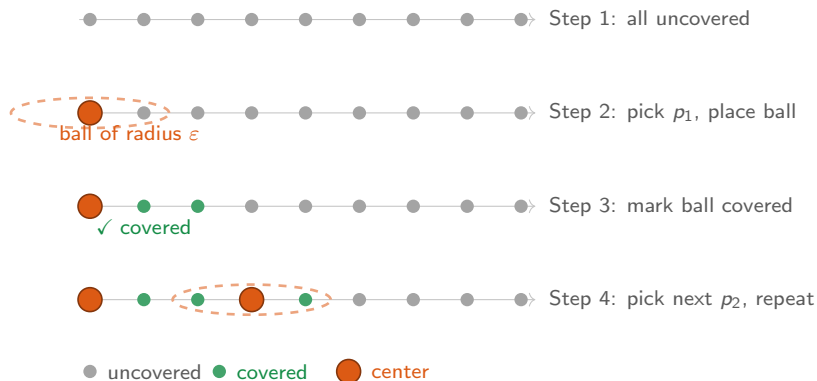
Place as few umbrellas as possible so that **every data point** is under at least one umbrella.

The greedy rule:

- 1 All points start **uncovered**.
- 2 Pick the **first** uncovered point p .
Place a ball of radius ε around p .
- 3 Every point inside that ball is now **covered** by p .
- 4 Repeat from step 2 until **all points are covered**.

Result: a list of **ball centers** and, for each data point, the list of centers that cover it.

Algorithm 1 – Greedy Cover: Step-by-Step



Algorithm 2 – Max-Min Cover: Concept

Same task, smarter strategy

Place balls so that centers are **spread as evenly as possible** across the data.

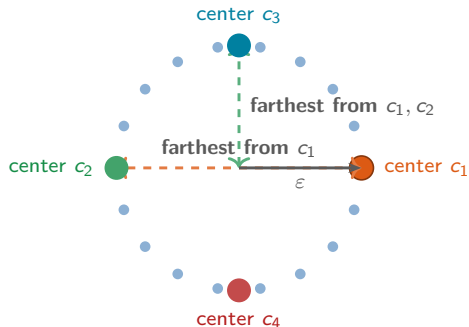
The max-min rule:

- 1 Start with **any** point as the first center.
- 2 Find the point that is **farthest away** from all existing centers.
- 3 Add it as a new center.
- 4 Repeat until the farthest distance $\leq \varepsilon$.
- 5 Assign every point to its nearby centers.

Advantage over Algorithm 1

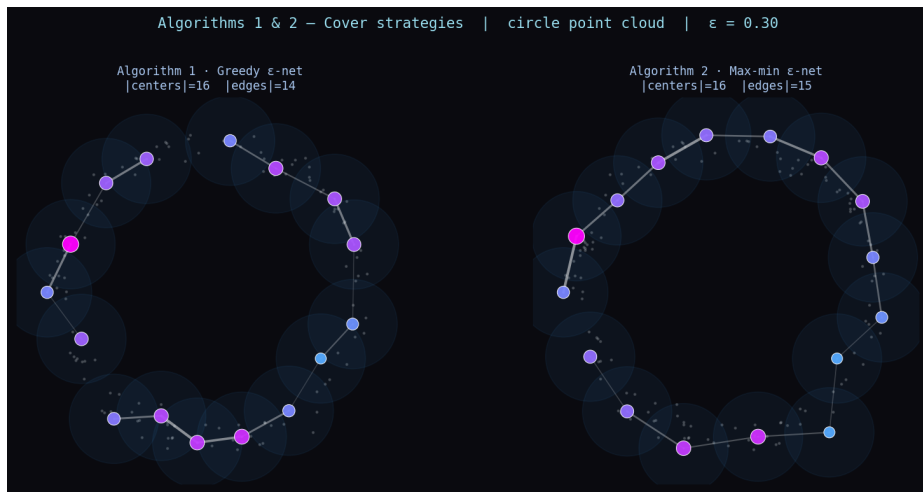
Centers are evenly spread $\rightarrow$ more uniform cover.
Can also stop after a fixed number of centers.

Algorithm 2 – Max-Min Cover: Visualization



Result: 4 evenly spread centers around the ring

Algorithms 1 & 2 – Side by Side on Real Data



How to read this visualization

Node colour = how many data points the ball covers (bright pink = many, dark blue = few).

Faint circles = the exact ϵ -balls around each center.

Algorithms 1 & 2 – Comparative Analysis

Left – Algorithm 1 (Greedy)

Centers appear wherever the scan *happened to start*.

Some small gaps, some slight clustering – but all points covered.

Summary: 16 centers, 14 edges.

Right – Algorithm 2 (Max-Min)

Centers are placed **as far apart as possible**.

The cover is more uniform – the ring looks cleaner.

Summary: Same number of centers, 15 edges.

Algorithm 3 – Building the Graph: Concept

Input

The list of centers and, for each data point, the list of centers that cover it.

Two simple rules:

- One **node** in the graph per ball center.
- An **edge** between two centers if at least one data point lies inside *both* their balls.
- Edge **thickness** = how many shared points (more shared → thicker line).

Why this captures shape

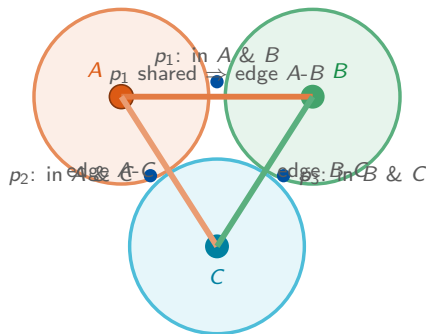
Nearby data regions → overlapping balls → edges.

The graph **mirrors the topology** of the data.

A loop in the data → a loop in the graph.

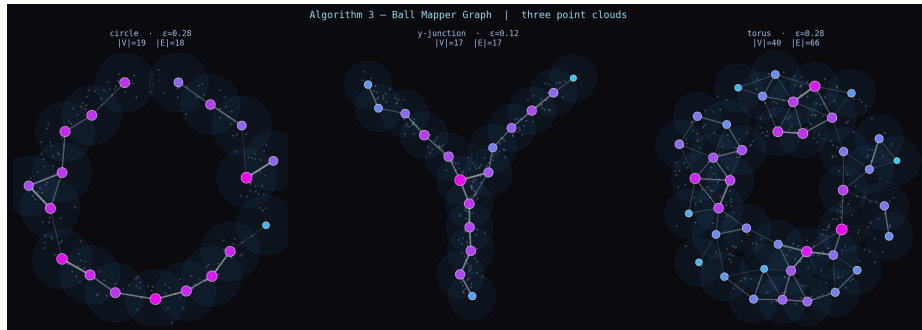
A branch in the data → a branch in the graph.

Algorithm 3 – Building the Graph: Intersection Logic



Result: triangle graph = 3 overlapping balls

Algorithm 3 – Three Different Point Clouds



Circle (left) $\epsilon=0.28$

Data on a ring $\rightarrow$ BM graph is a **loop**.

19 nodes, 18 edges.

The circular structure is perfectly preserved.

Y-Junction (middle) $\epsilon=0.12$

Data from a Y-shape $\rightarrow$ BM graph is a **Y**.

17 nodes, 17 edges.

The branching point appears as a high-degree node.

Torus (right) $\epsilon=0.28$

Data from a donut surface $\rightarrow$ **grid-like ring**.

40 nodes, 66 edges.

Many neighbours per node confirms local $\dim = 2$.

Algorithm 4 – Multi-Scale: Concept

The problem

How do you choose the right ε ?

- **Too small:** hundreds of tiny nodes, too much detail.
- **Too large:** everything merges into a blob.

The solution

Use many ε values at once.

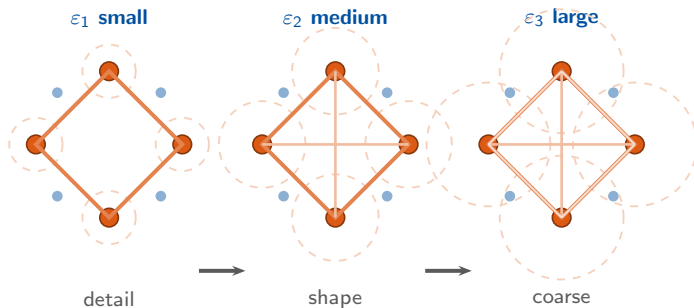
- ① Fix the centers using the **smallest** ε_1 .
- ② Keep the **same centers**, grow the radius.
- ③ Build one graph per scale.

Key guarantee

A feature visible at both ε and 2ε is **real structure**.

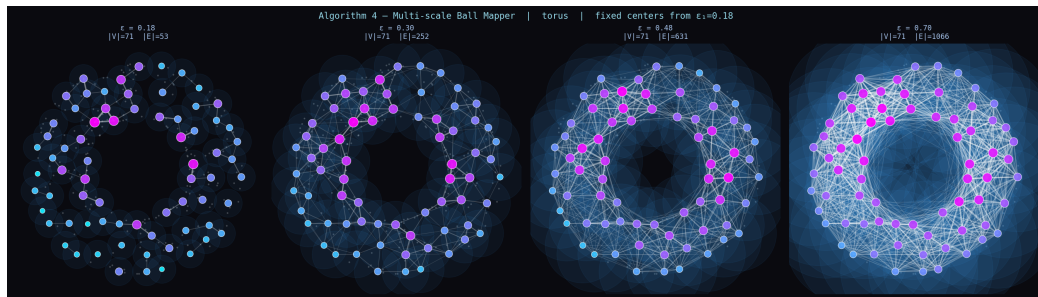
A feature that disappears quickly is noise.

Algorithm 4 – Multi-Scale: Visualizing Growing Radius



Same centers in all three panels – only ϵ grows

Algorithm 4 – Multi-Scale Analysis on the Torus



Methodology: The Fixed-Center Rule

Constraint: All 4 panels share the same **71 nodes** (centers are fixed at the smallest radius, $\varepsilon_1 = 0.18$).

Process: As we grow the radius ε from $0.18 \rightarrow 0.70$, the nodes remain static while the connectivity (edges) evolves. This reveals which structures are "stable" across different scales.

Multi-Scale Ball Mapper: Scale Comparison

$\varepsilon = 0.18$	53 edges	$\varepsilon = 0.30$	252 edges	$\varepsilon = 0.48$	631 edges	$\varepsilon = 0.70$	1066 edges
Fine detail. Many small overlaps. Local structure is noisy; global shape not yet clear.		Ring of rings visible. The characteristic donut topology appears clearly.		Inner and outer rings both visible. Robust connectivity across the entire shape.		Very dense. Local detail is lost. Only the broad global shape remains.	

The Stable Window

The most informative topological representation of this dataset is found in the middle scales (**Panels 2–3**), where the shape is clear but not yet over-saturated.

Dimension Estimation – The Core Concept

The key observation

How many **neighbouring nodes** does a typical node have?

This number depends on the **dimension** of the data:

- On a 1D line: ≈ 2 neighbours (left and right).
- On a 2D surface: ≈ 6 neighbours (like a hex grid).
- In 3D: many more.

How it is measured:

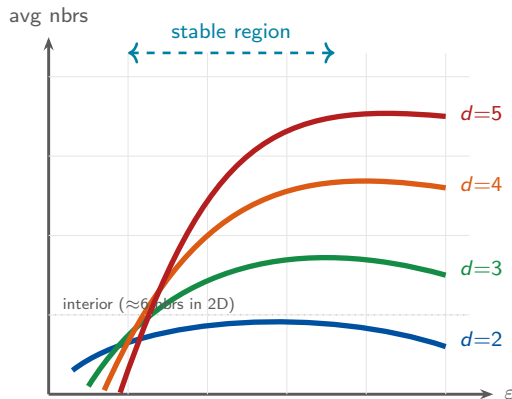
- 1 Build the BM graph for many values of ε .
- 2 Compute the **average number of neighbours** per node.
- 3 Plot: different dimensions give **separated curves**.

Important caveat

This gives an **upper bound** only.

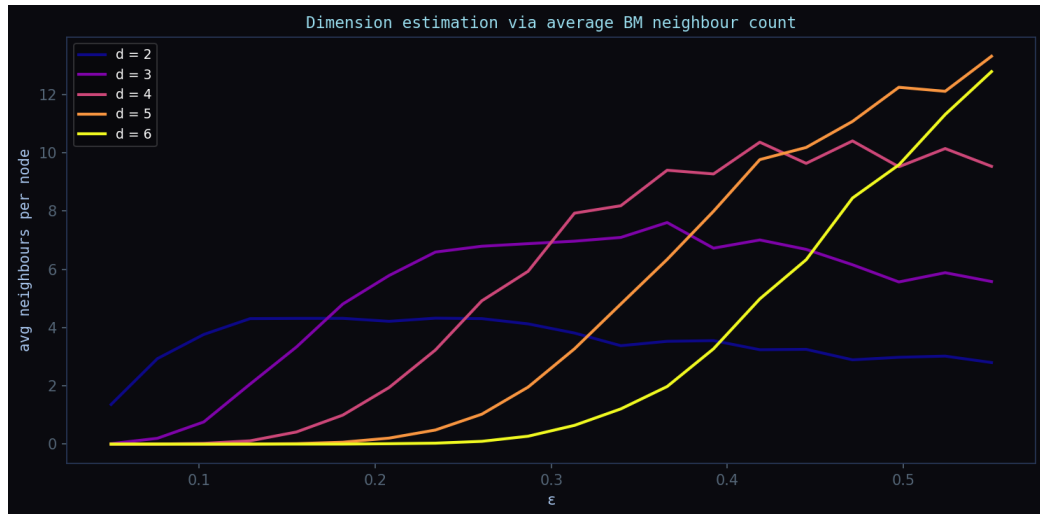
A truly high-dim dataset *cannot* appear low-dim.

Dimension Estimation – Curve Visualization



Curves separated by dimension = dimension is detectable

Dimension Estimation – Visual Analysis



Dimension Estimation – Reading the Plot

The Legend and Metrics

Visual Key: Each coloured curve represents one dimension $d \in \{2, 3, 4, 5, 6\}$.

- **X-axis:** The ball radius ε .
- **Y-axis:** Average number of neighbours per node.
- **Trend:** Higher dimension $\rightarrow$ more neighbours $\rightarrow$ **higher curve**.

The curves are clearly **separated**, allowing the intrinsic dimension to be read directly from the graph.

- **Note:** As ε grows, the "volume" available for overlaps increases faster in higher dimensions, causing the vertical separation.

Dimension Estimation – Interpreting Curve Behavior

Why the curves are not flat

The relationship between ε and connectivity changes across scales:

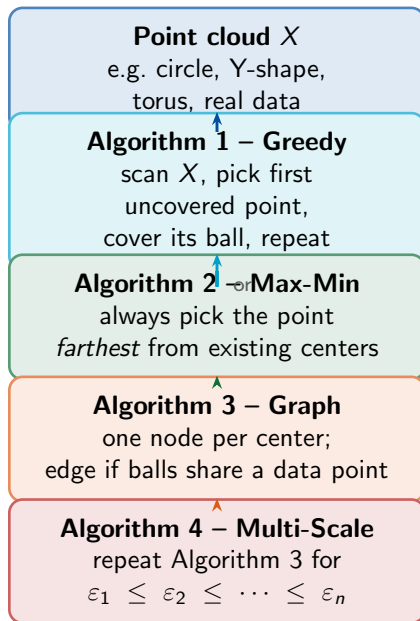
- **Very small** ε : Balls are too tiny to overlap significantly $\Rightarrow$ few neighbours.
- **Very large** ε : All balls overlap with almost every other ball $\Rightarrow$ connectivity saturates.

Finding the Truth: The Stable Region

The "truth" about the data's dimension is found in the **stable region** (the middle section where the curves level out).

Goal: Identify the flat plateau in the plot; this provides the most accurate estimate for the intrinsic dimension d .

Putting It All Together: The Pipeline



Putting It All Together: Outputs & Takeaways

What each step gives

Step	Output
Alg. 1 or 2	Ball centers + cover vector
Alg. 3	Ball Mapper graph $G = (V, E)$
Alg. 4	Multi-scale shape graphs (one per scale)

Shape is preserved

- Circle in data → **loop** in graph
- Y-shape in data → **Y-junction** in graph
- Torus in data → **ring of rings** in graph

Only one parameter

The radius ε controls everything.

No complex lens functions or external clustering algorithms needed.

Summary – What We Showed

Four Steps, One Idea

- ① **Greedy cover** – fast approach, scanning data sequentially.
- ② **Max-Min cover** – smarter strategy to spread centers evenly.
- ③ **Build graph** – connectivity defined by shared data points.
- ④ **Multi-scale** – ability to zoom in and out by varying ε .

Experimental Results

- **Fig. 1:** Consistency across different cover strategies.
- **Fig. 2:** Verified shape preservation (Circle, Y, Torus).
- **Fig. 3:** Demonstrated the transition from detail to coarse.
- **Fig. 4:** Proved dimension d is detectable via neighbour counts.

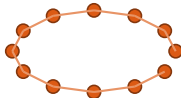
Summary – The Big Picture

Data (1300 points, torus)



Ball Mapper (ε)

Graph (40 nodes, torus shape)



Shape preserved, only 1 parameter needed

Code Implementation

github.com/is9008/Ball-Mapper-Topological-Data-Analysis-

Literature Source

Paweł Dłotko, *Ball Mapper: a shape summary for topological data analysis*,
arXiv:1901.07410v1 [math.AT], 2019. [cite: 180]

Thank you!

Questions?